CP4 - DBS - Transakce a trigger

Michal Hrouda a Lukáš Hrubec

v1.0
25. 4. 2026

Obsah

Transakce 2
Vytvoření a použití pohledu 3
Vytvoření a použití triggeru 4
Index 5

Transakce

Tato transakce řeší proces bezpečné výměny manažera v konkrétním oddělení. Operace zahrnuje odstranění stávajícího záznamu o manažerovi a okamžité jmenování nového zaměstnance do této role. Využíváme úroveň izolace SERIALIZABLE. Konflikt: Pokud by dva administrátoři prováděli výměnu manažera ve stejném oddělení současně bez použití transakce, mohlo by dojít k situaci, kdy by se systém snažil vložit dva záznamy se stejným id_oddeleni, což by vyvolalo chybu.

```
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;

DELETE FROM "Manazer"
WHERE "id_oddeleni" = 2;

INSERT INTO "Manazer" ("uroven_pravomoci", "id_zamestnanec", "id_oddeleni")
VALUES (4, 24, 2);
COMMIT;
```

```
2026-04-25 19:02:33] hroudmi5.public> BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE
2026-04-25 19:02:33] completed in 9 ms
2026-04-25 19:02:33] hroudmi5.public> DELETE FROM "Manazer"
                                      WHERE "id_oddeleni" = 2
2026-04-25 19:02:33] 1 row affected in 10 ms
2026-04-25 19:02:33] hroudmi5.public> INSERT INTO "Manazer" ("uroven_pravomoci", "id_zamestnanec", "id_oddeleni")
                                      VALUES (4, 24, 2)
2026-04-25 19:02:33] 1 row affected in 11 ms
2026-04-25 19:02:33] hroudmi5.public> COMMIT
2026-04-25 19:02:33] completed in 13 ms
```

	id_manazer	uroven_pravomoci	id_zamestnanec	id_oddeleni
1	12	4	24	2

Obrázek 1: Využití transakce

Vytvoření a použití pohledu

Pohled v_kontaktni_seznam_manazeru slouží jako virtuální tabulka, která v reálném čase agreguje data z tabulek „Oddeleni“, „Manazer“, „Zamestnanec“ a „Telefon“. Uživatel (např. recepční nebo ředitel) nemusí zadávat složité dozazy. Stačí mu zavolat jeden jednoduchý dotaz nad tímto pohledem. Pohled neobsahuje citlivé údaje jako rodná čísla nebo data nástupu, které jsou uloženy v tabulce „Zamestnanec“, čímž napomáhá dodržování zásad GDPR.

```sql
CREATE VIEW v_kontaktni_seznam_manazeru AS
SELECT
    o.nazev AS oddeleni,
    z.jmeno || ' ' || z.prijmeni AS manazer,
    z.email,
    t.telefon
FROM "Oddeleni" o
JOIN "Manazer" m ON o.id_oddeleni = m.id_oddeleni
JOIN "Zamestnanec" z ON m.id_zamestnanec = z.id_zamestnanec
LEFT JOIN "Telefon" t ON z.id_zamestnanec = t.id_zamestnanec
ORDER BY o.nazev;
```

	oddeleni	manazer	email	telefon
1	Analyza	Vojtech Hajek	vojtech.hajek8@firma.cz	+420 772 789 197
2	Finance	Michal Dvorak	michal.dvorak3@firma.cz	+420 627 790 745
3	HR	Adam Kral	adam.kral2@firma.cz	+420 778 825 821
4	Infrastruktura	Lenka Dvorak	lenka.dvorak7@firma.cz	+420 740 979 941
5	Marketing	Anna Zeman	anna.zeman4@firma.cz	+420 783 433 109
6	Obchod	Katerina Novotny	katerina.novotny10@firma.cz	+420 712 132 113
7	Podpora	Lukas Novak	lukas.novak9@firma.cz	+420 794 303 772
8	Testovani	Vojtech Vesely	vojtech.vesely6@firma.cz	+420 670 768 742
9	Vedeni	Simona Bartos	simona.bartos5@firma.cz	+420 677 383 847
10	Vyvoj	Klara Novotny	klara.novotny1@firma.cz	+420 632 574 343

Obrázek 2: Ukázka výstupu view

Vytvoření a použití triggeru

Pro zajištění konzistence dat v modulu docházky jsme navrhli trigger trg_kontrola_dochazky. Tento spouštěč se aktivuje automaticky před každým vložením (BEFORE INSERT) nového řádku do tabulky „ZaznamDochazky“. Jeho úkolem je zkontrolovat, zda se zaměstnanec nesnaží přihlásit (zadat nový čas příchodu) v situaci, kdy ještě neuzavřel svůj předchozí záznam (tzn. v databázi existuje jeho předchozí záznam, kde je cas_odchodu roven NULL). Pokud takový neuzavřený záznam existuje, trigger vložení zablokuje a vyhodí chybovou hlášku. Zabráníme tím vzniku překrývajících se a nelogických docházkových bloků.

```
CREATE FUNCTION kontrola_nedokoncene_dochazky()
RETURNS TRIGGER AS $$
BEGIN
    IF EXISTS (
        SELECT 1 FROM "ZaznamDochazky"
        WHERE "id_zamestnanec" = NEW."id_zamestnanec"
          AND "cas_odchodu" IS NULL
    ) THEN
        RAISE EXCEPTION 'Chyba: Zaměstnanec (ID %) se nemůže znovu přihlásit,
dokud neukončí předchozí docházku!', NEW."id_zamestnanec";
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_kontrola_dochazky BEFORE INSERT ON "ZaznamDochazky"
FOR EACH ROW EXECUTE FUNCTION kontrola_nedokoncene_dochazky();
```

SQL 4: Vytvoření triggeru

```
2026-04-25 18:51:26] hroudmi5.public> CREATE TRIGGER trg_kontrola_dochazky BEFORE INSERT ON "ZaznamDochazky"
                                      FOR EACH ROW EXECUTE FUNCTION kontrola_nedokoncene_dochazky()
2026-04-25 18:51:26] completed in 13 ms
2026-04-25 18:52:51] hroudmi5.public> INSERT INTO "ZaznamDochazky" ("cas_prichodu", "cas_odchodu", "typ_zaznamu", "id_zamestnanec")
                                      VALUES ('2062-04-02 10:00:00.', NULL, 'Běžná pracovní doba', 1)
2026-04-25 18:52:51] 1 row affected in 19 ms
2026-04-25 18:53:09] hroudmi5.public> INSERT INTO "ZaznamDochazky" ("cas_prichodu", "cas_odchodu", "typ_zaznamu", "id_zamestnanec")
                                      VALUES ('2062-04-03 10:00:00.', NULL, 'Běžná pracovní doba', 1)
2026-04-25 18:53:09] [P0001] ERROR: Chyba: Zaměstnanec (ID 1) se nemůže znovu přihlásit, dokud neukončí předchozí docházku!
2026-04-25 18:53:09] Kde: PL/pgSQL function kontrola_nedokoncene_dochazky() line 8 at RAISE
```

Obrázek 3: Vytvoření a ukázka funkčnosti triggeru

Index

V systému pro evidenci docházky je nejčastější operací vyhledávání historie záznamů pro jednoho konkrétního zaměstnance (např. při měsíčním zpracování mezd nebo kontrole absencí). Před vytvořením indexu: Pokud personalista hledá docházku zaměstnance (např. s ID 5), databáze musí provést tzv. Sequential Scan. To znamená, že musí vzít tabulku, která může mít statisíce záznamů, a projít ji celou řádek po řádku, aby našla ty správné dny. S přibývajícími roky historie by se tento dotaz neustále zpomaloval. Po vytvoření indexu: Díky indexu si databáze vytvoří logický stromový rejstřík. Při spuštění stejného dotazu plánovač použije Index Scan. Databáze přesně ví, na kterém místě na disku leží záznamy daného zaměstnance, a sáhne přímo pro ně. Čas spuštění (Execution Time) klesne z desítek či stovek milisekund na zlomky milisekundy, a to bez ohledu na to, jak moc tabulka v budoucnu naroste.

```sql
CREATE INDEX idx_dochazka_zamestnanec
ON "ZaznamDochazky"("id_zamestnanec");
```

```
QUERY PLAN
Seq Scan on "ZaznamDochazky"  (cost=0.00..740.96 rows=695 width=34) (actual time=0.874..9.585 rows=700 loops=1)
  Filter: (id_zamestnanec = 5)
  Rows Removed by Filter: 34301
Planning Time: 0.216 ms
Execution Time: 9.679 ms
```

```
QUERY PLAN
Bitmap Heap Scan on "ZaznamDochazky"  (cost=9.66..320.32 rows=693 width=34) (actual time=0.154..0.582 rows=700…
  Recheck Cond: (id_zamestnanec = 5)
  Heap Blocks: exact=91
  ->  Bitmap Index Scan on idx_dochazka_zamestnanec  (cost=0.00..9.49 rows=693 width=0) (actual time=0.117..0.…
        Index Cond: (id_zamestnanec = 5)
Planning Time: 0.750 ms
Execution Time: 0.678 ms
```

Obrázek 4: Ukázka funkčnosti indexu